

LeetCode 134: Gas Station

Authored by Arpit Raj, LNMIIT Jaipur • July 11, 2026

COMPANIES: BNY MELLON, DREAM11, FLIPKART, FREECHARGE

Problem Overview

There are n gas stations along a circular route, where the amount of gas at the i^{th} station is $gas[i]$. You have a car with an unlimited gas tank, and it costs $cost[i]$ of gas to travel from the i^{th} station to its next $(i + 1)^{th}$ station. You begin the journey with an empty tank at one of the gas stations.

Goal: Return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Approach 1: Brute Force

The brute force approach simply simulates the journey starting from every single gas station. For each starting point, we simulate moving to the next station and keep track of the current fuel. If the fuel ever becomes negative, it means this starting point is invalid, and we break out to try the next starting point.

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int n = gas.size();
        for(int start = 0; start < n; start++){
            int fuel = 0;
            bool ok = true;
            for(int i = 0; i < n; i++){
                int idx = (start + i) % n;
                fuel += gas[idx];
                fuel -= cost[idx];
                if(fuel < 0){
                    ok = false;
                    break;
                }
            }
            if(ok){
                return start;
            }
        }
        return -1;
    }
};
```

Complexity: Time complexity is $O(N^2)$ in the worst case, as for each station we might iterate through all other stations before failing. Space complexity is $O(1)$.

Approach 2: Greedy Observation (Optimal)

Key Mathematical Observations

From the handwritten notes, two critical observations enable a highly optimized greedy solution:

Observation 1: Feasibility

If the total amount of gas available across all stations is less than the total cost to travel the entire circuit, it is physically impossible to complete the journey, no matter where you start.

$$\text{If } \Sigma \text{ gas} < \Sigma \text{ cost} \Rightarrow \text{Return } -1$$

Observation 2: Eliminating Sub-paths

Let's define the current gain at station i as $\text{gain} = \text{gas}[i] - \text{cost}[i]$.

Suppose you start at station 8 and successfully reach station 66, but when attempting to reach station 67, your fuel becomes negative. This means **starting from 8 failed**.

Crucially, **none of the stations between 8 and 67 can serve as a valid starting point either**. For example, if you were to start at station 9 instead, you would skip the excess fuel you built up starting from station 8, making it even harder to reach station 67. Since the cost remains the same, if you couldn't reach 67 with the extra buffer from 8, you certainly cannot reach it starting from 9, 10, or any station up to 66.

- If $\text{current_gain} < 0$ at station i
- Every station from the *current start* up to i is impossible.
- Therefore, we can safely reset $\text{current_gain} = 0$ and update our potential start to $\text{start} = i + 1$, skipping all intermediate stations.

Greedy Implementation

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int n=gas.size();
        int gain=0;
        int start=0;
        int gsum=accumulate(gas.begin(),gas.end(),0);
        int csum=accumulate(cost.begin(),cost.end(),0);
        if(gsum<csum) return -1;

        for(int i=0;i<n;i++){
            gain+=gas[i]-cost[i];
            if(gain<0){
                start=i+1;
                gain=0;
            }
        }
        return start;
    }
};
```

Complexity: Time complexity is $O(N)$ because we only traverse the arrays a constant number of times (one pass to check total sum, one pass to find the starting index). Space complexity is $O(1)$.